

5-Mark Questions & Answers -- Data Structures Module 1

Topic 1: Concept of ADT

Q1. Explain ADT in detail. How is an ADT implemented in a programming language Give an example of Stack ADT with its operations.

1. Detailed Concept of ADT

2. Implementation in Programming Languages

3. Stack ADT Example & Operations

```
// Interface Definition (Stack ADT)
typedef struct Stack {
    int items[100];
    int top;
} Stack;

// Core Operations:
void push(Stack *s, int val) {
    if (s->top == 99) return; // Overflow
    s->items[++(s->top)] = val;
}

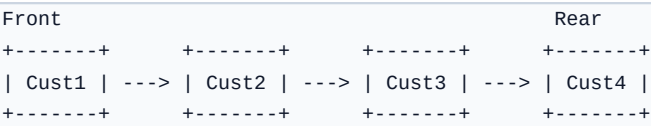
int pop(Stack *s) {
    if (s->top == -1) return -1; // Underflow
    return s->items[(s->top)--];
}

int peek(Stack *s) {
    return (s->top != -1) ? s->items[s->top] : -1;
}
```

Q2. Describe how ADT provides abstraction and information hiding with a real-life example (e.g., List ADT or Queue ADT).

1. Abstraction and Information Hiding

2. Real-Life Example: Queue ADT (Ticket Counter Analogy)

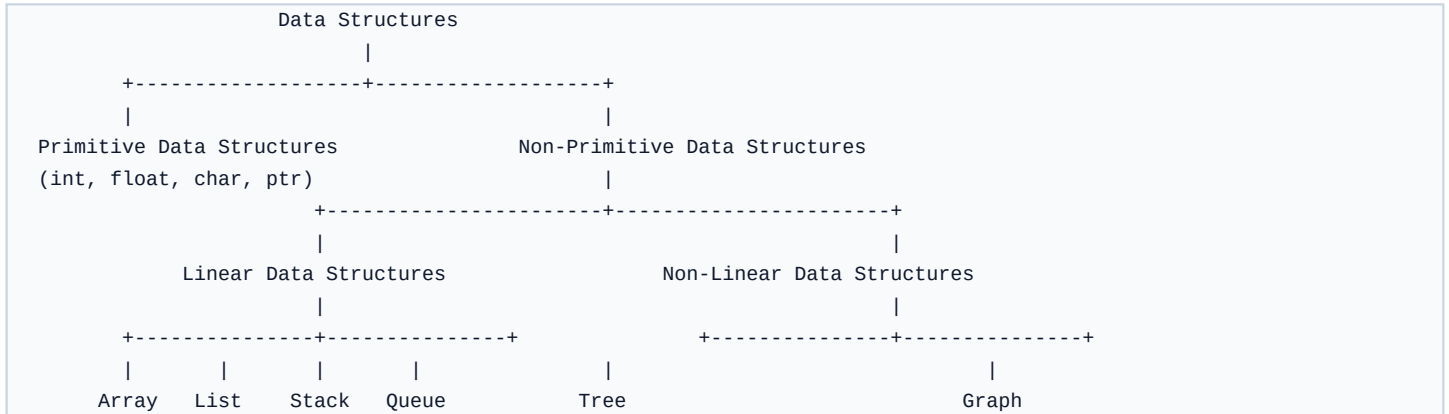


* Benefits:

Topic 2: Types of Data Structures

Q3. Explain the classification of data structures in detail with examples and diagrams.

Data structures are categorized into two primary hierarchies:



1. Primitive Data Structures

Base d

2. Non-Primitive Data Structures

Comple

Q4. Discuss the various types of data structures along with their real-world applications.

Data Structure	Type	Key Characteristics	Real-World Application
Array	Linear, Static	O(1) random access, contiguous memory.	Storing image pixels, tabular data.
Linked List	Linear, Dynamic	O(1) insertion/deletion without memory shift.	Undo functionality in text editors, Music playlist.
Stack	Linear, LIFO	Push/Pop at top only.	Function call stack, Browser back button.
Queue	Linear, FIFO	Enqueue at rear, Dequeue at front.	CPU process scheduling, Printer job queue.
Binary Tree / BST	Non-Linear	Hierarchical parent-child nodes.	Database indexing (B-Trees), File system.
Graph	Non-Linear	Vertices connected by edges.	Social network friend connections, GPS navigation.

Topic 3: Operations on Data Structures

Q5. Explain in detail the various operations performed on data structures: Traversing, Searching, Insertion, Deletion, Sorting, and Merging, with suitable examples for each.

1. Traversing

Access

2. Searching

3. Insertion

4. Deletion

5. Sorting

6. Merging

Q6. Discuss time complexity considerations for basic operations on data structures.

Time complexity measures how algorithm execution time grows as input size n increases:

Data Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)$ (at head)	$O(1)$ (at head)
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Binary Search Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

* Trade-off Analysis: Arrays offer instant $O(1)$ random access by index but suffer slow $O(n)$ insertions. Linked Lists sacrifice random access ($O(n)$) to achieve fast $O(1)$ dynamic insertion and deletion.

Topic 4: Arrays & Multi-Dimensional Arrays

Q7. Explain in detail how a 2-D array is represented in memory using row-major and column-major order. Derive the address calculation formula for both.

A 2-D array $A[M][N]$ with lower bounds LBR, LBC, Base Address B, and element size w :

1. Row-Major Order (RMO)

Equation:

2. Column-Major Order (CMO)

Equation:

Q8. What is a sparse matrix Explain its representation using triplet/array representation with an example.

1. Definition

2. Triplet Representation

Example Matrix (4 x 4):

Equation:

Triplet Table:

4	**4**	**3**	*(Total Rows, Total Cols, Non-Zero Count)
0	2	3	**A[0][2] = 3**
2	1	5	**A[2][1] = 5**
3	3	9	**A[3][3] = 9**

Q9. Explain multi-dimensional arrays with memory representation and address calculation with a numerical example.

1. Numerical Problem

Find the address of A[15][40].

2. Given Data

3. Calculation

Equation:

Equation:

Equation:

Equation:

Answer: Memory Address of A[15][40] = 2460.

Topic 5: String Manipulation

Q10. Explain various string manipulation operations (concatenation, comparison, substring, reverse) with algorithms/examples.

1. Concatenation: Appends string S2 to end of S1.

Q11. Write and explain an algorithm for pattern matching (Naive/Brute-force method) with an example.

1. Naive Pattern Matching Algorithm

4.
5.
6.

2. Numerical Example

* Te

Q12. Discuss string storage representations (fixed length vs linked list representation) in detail.

Feature	Fixed-Length Array Representation	Variable Linked List Representation
Storage Structure	Contiguous static char array `char s[1`	Dynamic list of nodes containing `char
Memory Efficiency	Wastes unused array space if string is	Allocates exact memory needed per char
Insertion / Deletion	Requires $O(n)$ character shifting.	$O(1)$ pointer updating without shif
Access Speed	$O(1)$ fast indexed access `s[i]`.	$O(n)$ traversal to reach index i

Topic 6: Self-Referential Structures with Pointers

Q13. Explain self-referential structures in detail with syntax, memory representation, and a diagram showing how nodes are linked.

A Self-Referential Structure is a C structure that contains at least one pointer member pointing to a structure of the same type:

```
struct Node {  
    int data;           // Data Payload  
    struct Node *next; // Self-referential pointer  
};
```

Memory Layout Diagram:

* Node 1 at address 1000 stores value 10 and pointer 2000.

N
+-----
| Dat
+-----

* No

Q14. Write a program/algorithm to create and traverse a simple linked list using self-referential structures.

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Node {  
    int data;  
    struct Node *next;  
};  
  
void traverse(struct Node *head) {  
    struct Node *temp = head;  
    printf("Linked List: ");  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

```
int main() {
    // Dynamic allocation of 3 nodes
    struct Node *head = (struct Node*)malloc(sizeof(struct Node));
    struct Node *second = (struct Node*)malloc(sizeof(struct Node));
    struct Node *third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 100;
    head->next = second;

    second->data = 200;
    second->next = third;

    third->data = 300;
    third->next = NULL;

    traverse(head);
    return 0;
}
```

Q15. Discuss the advantages of self-referential structures over arrays for dynamic data handling.

1. Dynamic Memory Allocation: Memory is allocated at runtime as needed via malloc(), eliminating static array capacity limits and memory wastage.

2. E
arrays